

CS247 – Assignment 5

Mustafa Albahrani

1 Setup (macOS)

The starter assumed Windows: `CMakeLists.txt` linked `opengl32` and the shaders used `#version 450`, which is not supported by macOS' OpenGL implementation (max 4.1). The following changes were made to get the framework running:

- Replaced `opengl32` with `find_package(OpenGL) / OpenGL::GL` so it links cross-platform.
- Requested a Core 4.1 forward-compatible context with `glfwWindowHint`.
- Lowered shaders to `#version 410`.
- Replaced the deprecated `GL_INTENSITY16 / GL_LUMINANCE` texture format with `GL_R16 / GL_RED`, with the fragment shader splatting `.r` into RGB for grayscale display.
- Added `__APPLE__` branches in path strings since the macOS build places the binary directly under `build/` rather than `build/Debug/`.

2 Implementation

2.1 Glyph Visualization

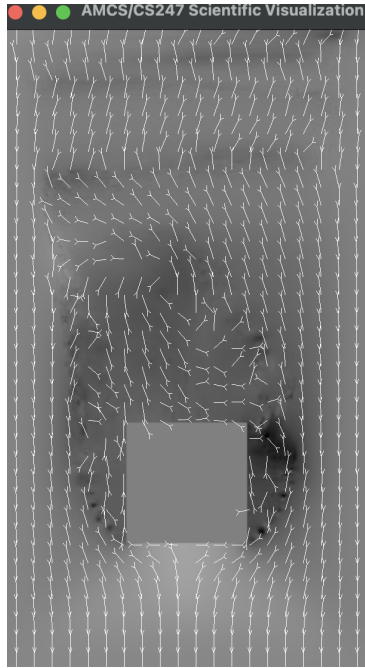
`drawGlyphs()` iterates the grid with a stride of `sampling_rate` cells. At each sample point a bilinear interpolation of the vector field is computed; the arrow is built from three line segments (shaft + two arrowhead segments) in normalized device coordinates and uploaded to a dynamic VBO drawn with `GL_LINES`. Sampling rate is adjustable (+/-) and the arrow length can be either constant or scaled by the local vector magnitude (`L`).

2.2 Streamlines

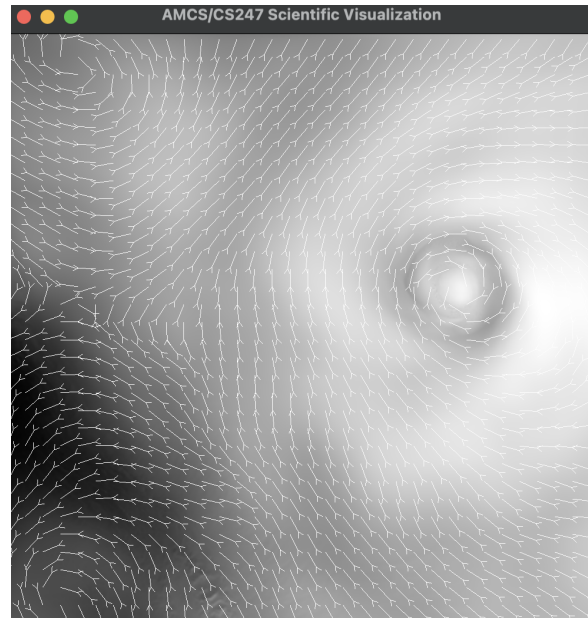
`computeStreamline(x,y)` integrates curves tangent to the instantaneous vector field. From each seed the curve is integrated both forward and backward (so the seed is in the middle, not at the end). Three integrators are available, switchable with `M`:

- Euler: $\mathbf{p}_{n+1} = \mathbf{p}_n + h \mathbf{v}(\mathbf{p}_n)$
- RK2 (midpoint): one half-step probe then a corrected step.
- RK4: classical four-stage scheme.

Vectors are sampled via bilinear interpolation, and integration stops when the curve leaves the grid, the vector magnitude falls below 10^{-5} , or the accumulated arc length exceeds a cap. Seeds are stored and the streamlines are recomputed whenever the timestep changes (0).



(a) Block dataset



(b) Hurricane dataset

Figure 1: Glyph visualization.

2.3 Pathlines

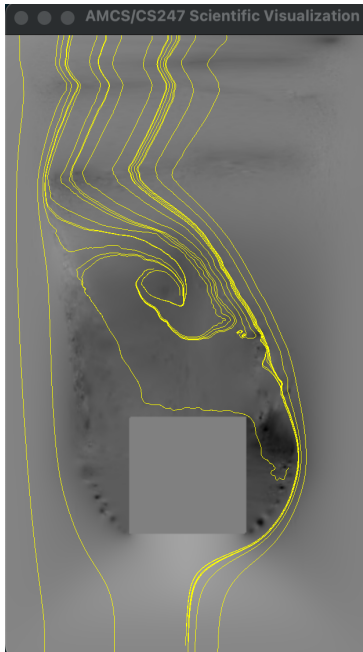
`computePathline(x,y,t)` uses the same three integrators but advances in both space *and* time, sampling the vector field via trilinear interpolation (bilinear in space, linear between two adjacent timesteps).

2.4 Color-coded Scalar Field

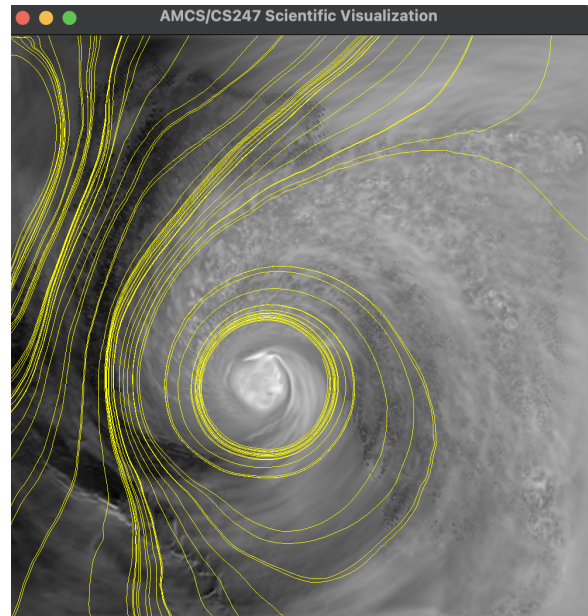
The fragment shader implements two colormaps applied to the scalar field texture: a rainbow ramp (blue $\rightarrow$ cyan $\rightarrow$ green $\rightarrow$ yellow $\rightarrow$ red via four `mix` segments) and a cool–warm ramp (blue $\rightarrow$ white $\rightarrow$ red). The user can dial color emphasis up or down (`[, ]`). Mode cycling is on `C`.

2.5 Bonus

- **RK4** integrator for both streamlines and pathlines (cycled with `M`).
- **Rake seeding (R)**: a single click releases seven streamline seeds spread along x at the clicked y .
- **Switching scalar fields (S)**: cycles between e.g. pressure and vorticity.



(a) Block dataset



(b) Hurricane dataset

Figure 2: Streamlines seeded interactively with the mouse.

3 Controls

1/2/3	block / tube / hurricane
0	next timestep
A, L, +/-	arrows, length mode, sampling rate
T, click	streamline seed
P, click	pathline seed
M	cycle Euler / RK2 / RK4
R	toggle rake seeding
X	clear all seeds
I/K	dt up / down
C, [/]	colormap, blend factor
S, B	scalar field, background

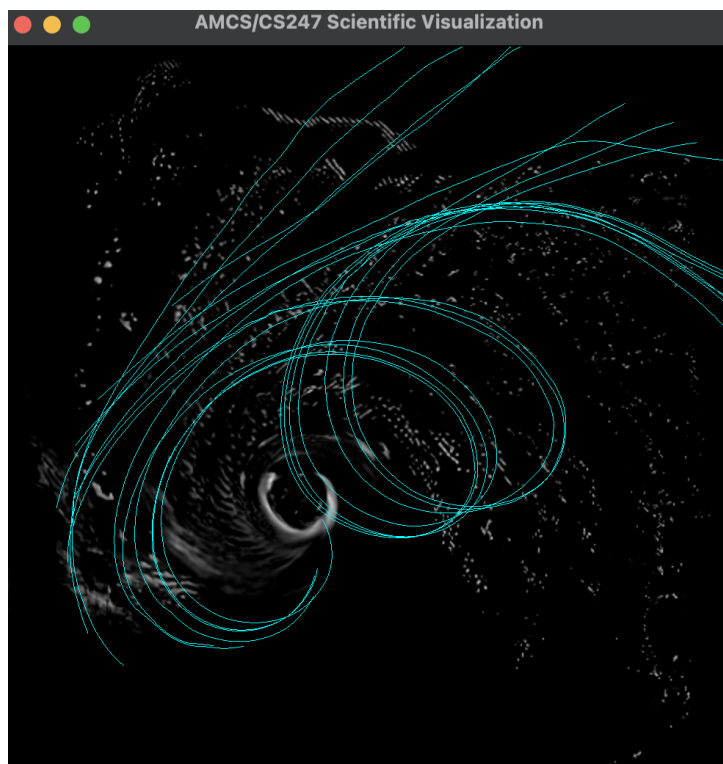


Figure 3: Pathlines on the hurricane dataset.